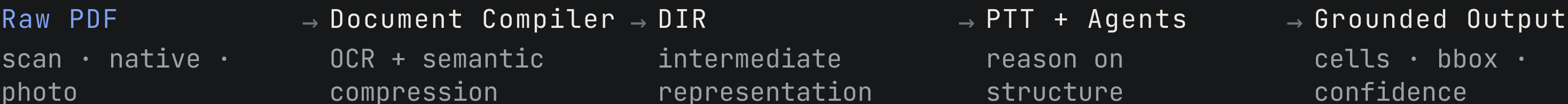


DocCompiler + PTT

Compile the document, then reason on it.

A document-compiler architecture with
Probabilistic Table Threading.



When OCR meets a page break, it guesses. [DocCompiler + PTT](#) knows.

TEAM NOOBDA
REVA University, Bengaluru

Today's IDP pipeline cracks at the **page boundary**

REAL SCENARIO – 40-PAGE BANK STATEMENT

A table starts on pg. 12, ends on pg. 18. Headers repeat every page; sub-totals every ten rows; scanner skew on pg. 15. The OCR stitcher assumes column 3 on pg. 12 equals column 3 on pg. 13 — wrong. The deposit on line 14 silently lands in the wrong row. **Loan funds. Borrower defaults.**

TODAY'S TYPICAL PIPELINE – AND WHERE EACH LAYER CRACKS

01 PDF Ingestion Scan skew, redactions and bleed-through ignored.	02 OCR Cells extracted in reading order, not logical order.	03 Table Detection Heuristic boundaries fail on novel layouts.	04 · breaks here Page Stitching Regex assumes column N on page X = column N on page X+1.	05 Output No grounding, no confidence — silent wrong values.
--	--	---	---	---

LLMs changed what's possible — but bolting GPT-4o onto this pipeline is **the horseless carriage with an engine strapped on**. The carriage itself needs redesigning.

What exists today — and exactly where it cracks

FinTabNet / PubTabNet — IBM, Microsoft STRONG Cell-level table-structure recognition on clean single-page tables. GAP Zero concept of page boundaries — a 30-page table never fits one pass.	LayoutLMv3 + Table Transformers STRONG Joint text + layout encoding; backbone of most commercial IDP. GAP 512-token window — no cross-page reasoning; re-tuned per doc type.
Traditional IDP — heuristic stitcher STRONG Deterministic, fast, auditable, cheap on clean consistent docs. GAP Hard-coded per template; silent misalignment on skew or header drift.	GPT-4o as document reader STRONG Reads messy scans and handwriting in natural language, no training. GAP Hallucinates row counts, no bboxes; 2000 pages ≈ \$400k/mo alone.

THE SHARED GAP No system reasons about [cross-page table identity as a structured inference problem](#), nor compiles documents into a stable intermediate representation before reasoning.

Two ideas nobody is combining

① Reason on a compiled representation
...not the raw document — like a compiler turning source into bytecode.

SOFTWARE WORLD	DOCUMENT WORLD
C++ source code	Raw PDF
Compiler	Document Compiler
Bytecode / IR	DIR
Runtime / CPU	PTT + Agents

② Stitch page boundaries as coreference
...not segmentation — so low-confidence edges fail loudly, not silently.

SEGMENTATION ASKS
“Where does this table end?”

COREFERENCE ASKS
“Do these two fragments share the same logical identity?”

Resolved by **Bayesian belief fusion over five independent signals** — detailed on the PTT slide.

DocCompiler + PTT, end to end

Raw PDF · scan / native / photo



DOCUMENT COMPILER		
OCR + Layout Analysis text · coords · tables	Semantic Compression 10k → 1.5k tokens	DIR Construction fragments + metadata

DIR — Document Intermediate Representation
fragment objects · table boundaries · row/cell metadata · coordinates



PTT BELIEF GRAPH		5-signal Bayesian fusion per fragment pair
P > 0.9 Auto-merge ≈ 85% of fragment pairs — no LLM	P < 0.9 LLM Arbiter ≈ 15% — escalated, structured judgment	



Verification Agent — balance checks · subtotal cross-check · anomaly flags



Grounded Output — threaded tables · cells · bbox · confidence

How raw PDFs become structured DIR objects

01 OCR + Layout Analysis

Extract text, bounding boxes and reading order — per cell (x, y, w, h) with a confidence score. We keep OCR because **visual grounding is non-negotiable for audit**.

02 Semantic Compression

Strip boilerplate before any LLM reasoning — page numbers, disclaimers, branch addresses out; tables, headers, values and entity relationships kept. **10,000 → ~1,500 tokens (85% reduction)**.

03 DIR Object

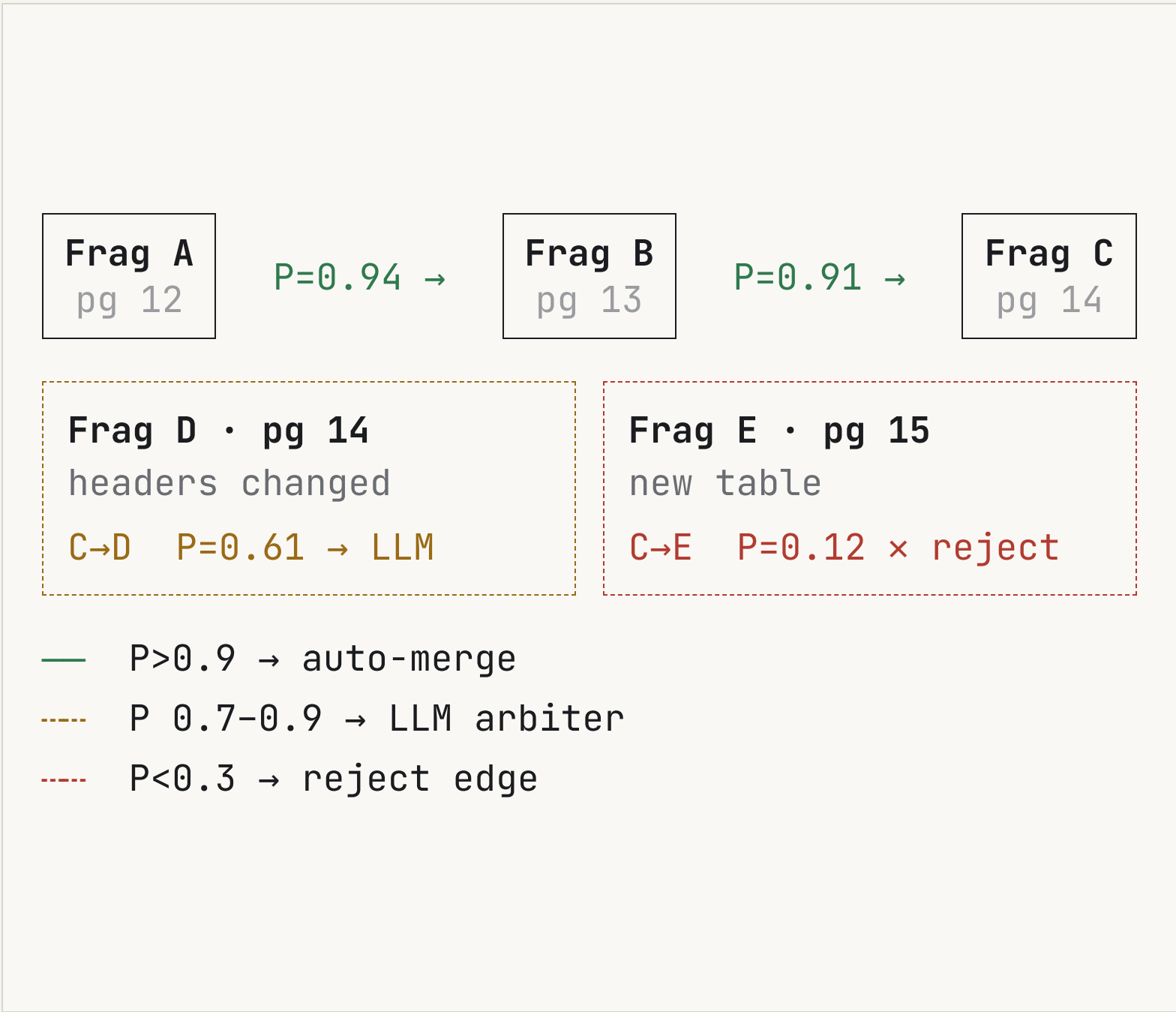
The compiler's output: a structured fragment with everything PTT needs to thread it — shown at right.

OUTPUT — ONE DIR FRAGMENT

```
{
  "fragment_id": "frag_012_A",
  "page": 12, "confidence": 0.97,
  "bbox": [120, 340, 580, 820],
  "headers": ["Date", "Description",
              "Amount", "Balance"],
  "column_fingerprint": [0.12, 0.35, 0.62, 0.84],
  "value_types": ["date", "text",
                  "currency", "currency"],
  "last_row_type": "data",
  "continues_to_next_page": null // PTT resolves
}
```

How DIR fragments get threaded into logical tables

BELIEF GRAPH – 40-PAGE BANK STATEMENT



THE FIVE EVIDENCE SIGNALS

- ① **Header similarity** — cosine over header-token embeddings. Survives reworded headers if widths match.
- ② **Column-width fingerprint** — normalised boundaries (0–1). Strongest structural signal; survives font/header drift.
- ③ **Value-type continuity** — date/currency/text consistent across the boundary? Type mismatch = strong negative.
- ④ **Spatial flow direction** — does A end near page bottom and B start near the next page's top?
- ⑤ **Subtotal pattern** — a subtotal row signals table end, not continuation — a negative signal.

One 40-page bank statement, step by step

6 tables · 2 span page boundaries · exactly 1 LLM call

01 Ingest + normalise Scan skew on pg. 15 detected and corrected; bleed-through denoised. Clean page images out.	02 Semantic compression Boilerplate stripped. 38,000 → 5,800 tokens.	03 DIR construction 6 fragments detected — each with headers, fingerprint, value types, per-cell bbox, page ref.
04 PTT graph built 5 signals over 5 adjacent pairs. A→B: P=0.94. C→D: P=0.61 → LLM.	05 LLM arbiter — 1 call Only 1 of 5 pairs escalates. Prompt: last row of pg. 13 + first row of pg. 14 → judgment + reason.	06 Verification + output End balance verified = opening + credits – debits. Every cell tagged: page + bbox + confidence.

Traditional IDP vs. DocCompiler + PTT

	TRADITIONAL IDP	DOCCOMPILER + PTT
Failure mode	Silent — bad data, no error	Loud — low-P edges flagged to a reviewer
LLM usage	Full doc → huge token cost	Post-compression, edge-only — 85% skip the LLM
Page boundary	Regex stitcher breaks on header drift	5-signal Bayesian fusion survives drift
Auditability	“Trust the pipeline” — no grounding	Every cell: page + bbox + confidence
Learning	Manual re-tune of heuristics	Corrections → few-shot bank → distillation
Novel layouts	Silent failure on unseen templates	LLM fallback + reviewer queue for new types
Cost at scale	Cheap but brittle on complex docs	≈ \$0.04/doc — 85% auto path, LLM only on hard edges

How week-1 performance becomes year-1 performance

WEEK 1 Heuristics + conservative thresholds • 5 signals, $P > 0.9$ to auto-merge. • All uncertain edges → reviewer queue; every correction logged. • Compression rules bootstrapped per document class. BASELINE CELL F1 ~80% clean · ~60% skewed	MONTH 1 Correction bank kicks in • 100+ corrections → few-shot RAG lookup. • Pairs checked vs. bank before any LLM call — 15% → 8%. • Compression refined per customer template. • Document fingerprinting — DIR hash vs. seen docs: exact match → cached output; partial → reprocess only changed fragments; same structure + changed values → fraud flag. ACCURACY ~90% on seen templates	YEAR 1 Distilled + self-extending • Lightweight Fragment Detector fine-tuned on collected pairs. • Per-doc-class signal-weight rulebook. • Novel layout → reviewer queue → added to bank. TARGET 95%+ known · graceful on unknowns
--	---	---

How we measure success — and what we're honest about

>0.90 Cell-level F1 per-cell correct, not table-level	>0.92 Row-boundary accuracy merged at the right row?	<15% LLM call rate cost proxy — edges escalated	<2% False merge rate different tables threaded as one
---	--	---	---

BENCHMARKED ON FinTabNet · DocLayNet · synthetic bank-statement generator (Faker + Jinja-to-PDF)

EXPLICIT TRADE-OFFS

Accuracy > coverage	Uncertain edge → reviewer, never a hallucination.
Latency / cost = balanced	85% auto path keeps p95 < 90s, < \$0.04/doc.
Grounding always	No bbox = no audit trail = no production use.
Deterministic first	5-signal scoring is deterministic; LLM is last resort.

WON'T CATCH — AND WE SAY SO

- Free-form text rows with no column structure
- Intentionally adversarial reformatting
- Handwritten tables on paper

WILL CATCH

Same document resubmitted with changed values — the DIR diff finds a structural match + value change = suspicious re-submission flag.

Four inputs and we build it right

01 Sample document set 20–30 anonymised multi-page bank statements and payslips with ground truth. Min. 5 where a table spans a page boundary.	02 Correction-log access Even 500 rows of “extracted X, correct Y, doc type Z” bootstraps the few-shot correction bank immediately.
03 LLM API budget Claude or GPT-4o key for ~2,000 calls — covers ~13,000 fragment pairs at our 15% escalation rate.	04 Infrd OCR output schema Raw per-page JSON/XML — bboxes, confidence, reading order. We build on your OCR; we don't replace it.

We are ready to build. We just need the data to build it right.